

SMART INDIA HACKATHON 2026

Problem Statement ID: SIH26156 | Category: Software | Theme: Miscellaneous

ULPF

Universal Log Pre-processing Framework

"Different Logs. One Framework. A Safer Tomorrow."

SOFTWARE REQUIREMENTS SPECIFICATION

SOFTWARE REQUIREMENTS SPECIFICATION

Functional, data and non-functional requirements baseline — revised to the Local Rule Authoring Agent architecture

Team & Role Allocation

Member	Role	Primary Ownership in This Document
Divyansh Rajat	Team Lead / System Architect	Overall architecture, scope, requirement baseline, traceability
Ashish Bhardwaj	Log Processing Engineer	Ingestion, fingerprinting, rule engine, parser factory
Sakshi	Normalization & Schema Engineer	Unified event model, OCSF/ECS mapping, rule registry, traceability
Parampreet Kaur	Backend & Integration Engineer	APIs, persistence, deployment, downstream connectors
Aryan Jeet	Frontend Engineer	Vendor portal (Studio, Rule Registry, Jobs & Sessions), dashboards
Saksham	Testing, Documentation & Demo	Test strategy, benchmark methodology, acceptance evidence

Reference Documents

Ref.	Document	Source
R1	SIH 2026 Problem Statement SIH26156 — Universal Log Preprocessing Framework	Smart India Hackathon 2026 portal
R2	ULPF Full Understanding & Reference Document (Team S.W.O.R.D.)	Internal design reference — governing document for this baseline
R3	OCSF — Open Cybersecurity Schema Framework	ocsf.io schema specification
R4	ECS — Elastic Common Schema field reference	Elastic documentation
R5	Qwen2.5-Coder technical report	Qwen team / Alibaba, open weights
R6	llama.cpp — local GGUF inference runtime	ggerganov/llama.cpp, open source
R7	RFC 5424 — The Syslog Protocol	IETF
R8	ArcSight Common Event Format (CEF) and IBM LEEF specifications	Vendor specifications
R9	OWASP — Regular expression Denial of Service (ReDoS)	OWASP documentation

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the functional and non-functional requirements for ULPF, the Universal Log Pre-processing Framework, developed by Team S.W.O.R.D. for Smart India Hackathon 2026 problem statement SIH26156. It is the authoritative requirement baseline for the development team and evaluators. It specifies what the system shall do; the accompanying Technical Requirements Document specifies how.

1.2 Scope of the Software

ULPF is a self-hosted, air-gapped preprocessing framework that receives perimeter security log events from heterogeneous sources, stores the exact raw event before any transformation, matches or drafts a versioned declarative parsing rule, deterministically extracts and normalizes fields into a common OCSF/ECS-aligned event model, and delivers standardised events to downstream security and analytics platforms.

ULPF shall not perform threat detection, correlation, alerting or case management. It shall not act as a long-term archival platform, an endpoint collection agent, or a multi-tenant hosted service. The Local Rule Authoring Agent shall never run in the production hot path and shall never be reachable outside onboarding.

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
CEF	Common Event Format — a text log format widely used for security events
Event ID	A unique identifier allocated at ingestion, linking the raw object, normalized outputs and processing history
Fingerprint	A key-set hash (structured logs) or structural/delimiter signature (unstructured logs) used to match a sample against Active rules
GGUF	A quantized model-weight file format used by llama.cpp for local LLM inference
Ingestion Job	A batch upload of many events assumed to share one format
Ingestion Session	A streaming connection (session_id on every event) assumed to share one format
LEEF	Log Event Extended Format — IBM's event format consumed by QRadar
OCSF	Open Cybersecurity Schema Framework — the primary target taxonomy
ECS	Elastic Common Schema — an adapter/rendering profile over the same normalized event
Parsing Rule	A versioned, declarative bundle: extraction pattern, field mapping, target schema and masking policy for one log format
Rule Authoring Agent	A local, quantized LLM (Qwen2.5-Coder-7B via GGUF/llama.cpp) invoked only on a registry miss, during onboarding
Rule Hash	SHA-256 of the canonical rule content, proving the exact configuration used to parse an event
Rule Registry	A versioned PostgreSQL store of rules, versions, test cases, approvals, fingerprints and audit history
Raw Event Vault / Immutable Raw Evidence Layer	MinIO/S3-compatible object storage holding the exact original event bytes, write-once, hashed at write time
Unmapped Fields	Vendor-specific data retained in the normalized output when no safe common-schema mapping exists

1.4 References

See the Reference Documents table in Document Control. Requirements derived directly from the problem statement are tagged PS; requirements derived from the ULPF Full Understanding & Reference Document are tagged UD; requirements derived from engineering analysis are tagged DER.

1.5 Document Conventions

- Shall denotes a mandatory requirement.
- Should denotes a recommended requirement whose omission requires justification.
- May denotes an optional capability.
- Priority is High (required for the prototype), Medium (required for a complete product), or Low (desirable).
- Requirement IDs are stable. FR-n.m identifies functional requirements; NFR-x.n identifies non-functional requirements; DR-n identifies data requirements; IR-n identifies interface requirements.

2. Overall Description

2.1 Product Perspective

ULPF is a new, self-contained middleware system. Its upstream boundary is the set of perimeter log-producing devices (firewalls, routers, VPN gateways, IDS/IPS, WAFs, proxies, cloud edge). Its downstream boundary is the set of consuming platforms: SIEMs, data lakes, ClickHouse/OpenSearch analytics, dashboards and REST clients.

The system is decomposed into eight logical subsystems:

#	Subsystem	Responsibility	FR group
S1	Ingestion Gateway	Accept events over Syslog, REST/gRPC, batch upload and streaming session transports; allocate event_id; bind to a registered, API-key-scoped source	FR-1
S2	Detection & Rule Registry	Fingerprint incoming samples; search Active rules; route a miss to the authoring agent; lock and spot-check batch/session rules	FR-2
S3	Local Rule Authoring Agent	Draft a declarative candidate rule via a local quantized LLM, onboarding-only, never in the hot path	FR-3
S4	Rule Engine (Deterministic Parsing & Normalization)	Execute the matched rule's parser, validate required fields/types, normalize to the canonical model and OCSF/ECS output	FR-4
S5	Immutable Raw Evidence Layer	Store the complete original event immutably with an integrity digest and reception metadata before any parsing	FR-5
S6	Traceability Layer	Bind event_id, raw hash, rule_id/version/hash and schema_version; serve bidirectional lookup	FR-6
S7	Rule Registry Governance	Hold versioned rules, fingerprints, test cases, approvals, lifecycle state and audit history	FR-7
S8	Vendor Portal, Review & Delivery	Studio onboarding, Rule Registry UI, Jobs & Sessions, API & Keys, Log Review, query API and downstream export	FR-8, FR-9

2.2 Product Functions — Summary

At the highest level the system performs the following functions in sequence for every event:

- Receive the event, allocate an event_id, and record it verbatim with reception metadata.
- Persist the exact raw bytes to the Immutable Raw Evidence Layer and compute a SHA-256 digest, before any parsing.
- Compute a format fingerprint and search Active rules in the Rule Registry (or use an explicit rule_id if supplied).
- On a registry miss during onboarding, invoke the Local Rule Authoring Agent to draft a candidate declarative rule.
- Validate the candidate structurally, compile it in a sandbox, run all test samples, and route it to Draft / Pending Review for mandatory human approval.
- For an Active rule, run the deterministic parser, validate required fields and types, and normalize into the canonical ULPF model, then OCSF and ECS.
- Retain any field with no safe mapping under unmapped_fields; never discard it.
- Record traceability metadata (event_id, raw hash, rule_id, rule_version, rule_hash, schema_version) and deliver the standardised event downstream.

2.3 User Classes and Characteristics

User class	Characteristics	Privileges required
Security Analyst	Consumes normalized events; occasional trace-back during investigation	Read events, read traceability metadata
Rule / Detection Engineer	Onboards sources via Studio; reviews and approves LLM-drafted rule candidates	All analyst rights plus approve/edit rules
Platform Administrator	Configures sources, transports, API keys and thresholds; operates the deployment	Full configuration and operational rights
Compliance / Audit Officer	Retrieves raw events and rule-version provenance as evidence	Read raw events, read audit trail
Vendor / Downstream System	Machine consumer retrieving normalized events via API, export or Kafka	Source-scoped API key, read/write limited to its own scope

2.4 Operating Environment

Aspect	Requirement
Host OS	Linux x86-64 (Ubuntu 22.04 LTS or equivalent); Windows via container runtime
Runtime	Container runtime with Compose support
Network	Operates fully within an isolated network; no outbound Internet access required at install or runtime, including for the Rule Authoring Agent
Minimum prototype hardware	4 vCPU, 8 GB RAM, 50 GB storage (CPU-only GGUF inference)
Recommended hardware	8 vCPU, 16 GB RAM, SSD storage, optional GPU for faster rule drafting
Accelerators	None required; the quantized 7B model shall run acceptably on CPU
Client	Modern evergreen browser for the vendor portal / Studio

2.5 Design and Implementation Constraints

- C1 — No component on the processing or authoring path shall require outbound Internet connectivity, a cloud API or an external inference service.

- C2 — All third-party components and model weights shall be open/licensed for redistribution in an offline bundle.
- C3 — The system shall be deployable as containers orchestrated by a single Compose definition.
- C4 — The core event schema shall align with OCSF as the primary taxonomy, with ECS as an adapter, rather than a proprietary schema.
- C5 — The Rule Authoring Agent's GGUF model weights shall be bundled at build time, not downloaded at runtime.
- C6 — Raw event storage shall be write-once; no processing path may modify a stored raw event.
- C7 — A parsing rule shall never contain arbitrary Python, shell commands, SQL, dynamic imports, URLs, credentials or network actions — only declarative, schema-validated content.
- C8 — All configuration shall be file-based and version-controllable; no configuration shall exist only in a database or UI state.

2.6 Assumptions and Dependencies

- A1 — Log events are text-based or textually representable; encrypted and binary proprietary payloads are out of scope.
- A2 — 3–5 representative samples are sufficient for the Rule Authoring Agent to draft a usable candidate rule.
- A3 — Reviewers with domain knowledge are available to adjudicate every Draft/Pending-Review candidate.
- A4 — The deployment host clock is synchronised; ULPF records its own receipt time independently of source timestamps.
- D1 — The system depends on a local quantized LLM runtime (llama.cpp + Qwen2.5-Coder-7B GGUF), PostgreSQL, MinIO/S3-compatible storage, ClickHouse, OpenSearch, Kafka/Redpanda and a container runtime.

3. External Interface Requirements

3.1 User Interfaces

ID	Requirement	Pri.
IR-1.1	The system shall provide a web-based vendor portal accessible over HTTPS, comprising Studio, Rule Registry, Jobs & Sessions, API & Keys and Log Review screens.	High
IR-1.2	Studio shall provide an onboarding wizard covering sample submission, fingerprinting, candidate rule review and approval.	High
IR-1.3	The console shall present the raw sample and the candidate rule's field mapping side by side with per-field linkage.	High
IR-1.4	The Jobs & Sessions screen shall show a live per-event strip (detecting → locked → spot-checked) and the currently locked rule_id@version.	High
IR-1.5	The API & Keys screen shall allow generating, scoping (per source/environment), rotating and revoking keys.	High
IR-1.6	The Log Review screen shall list unresolved, ambiguous and format-drift events for manual triage.	Medium
IR-1.7	The console shall be usable at 1366×768 and above and shall not require any plugin.	Medium

3.2 Software Interfaces

ID	Requirement	Pri.
IR-2.1	The system shall expose a versioned REST/gRPC API over HTTPS for conversion, rule management, job/session management and traceability retrieval.	High
IR-2.2	The API shall publish a machine-readable OpenAPI specification served by the system itself.	High
IR-2.3	The system shall interface with PostgreSQL for the Rule Registry, jobs/sessions, unresolved events and audit records.	High
IR-2.4	The system shall interface with MinIO/S3-compatible storage for the Immutable Raw Evidence Layer.	High
IR-2.5	The system shall interface with ClickHouse for normalized-event analytics and OpenSearch for full-text search.	Medium
IR-2.6	The system shall be able to produce to and, where configured, consume from a Kafka/Redpanda topic for asynchronous export.	Medium
IR-2.7	The system shall interface with a local llama.cpp inference server hosting the Qwen2.5-Coder-7B GGUF model, reachable only from the onboarding service.	High

3.3 Communications Interfaces

ID	Requirement	Pri.
IR-3.1	The system shall accept Syslog over UDP and TCP on configurable ports.	High
IR-3.2	The system shall accept single-event conversion over POST /v1/convert with an optional rule_id.	High
IR-3.3	The system shall accept batch conversion over POST /v1/jobs (file upload) with auto-detect-and-lock.	High
IR-3.4	The system shall accept streaming conversion over POST /v1/sessions/{id}/events with a session_id on every event.	High
IR-3.5	All externally exposed HTTP interfaces shall support TLS with configurable certificates.	High
IR-3.6	The system shall not initiate any outbound connection to a public network endpoint during normal operation, including from the Rule Authoring Agent.	High

3.4 Hardware Interfaces

ULPF has no direct hardware interface requirements. It interacts with host resources through the container runtime: network sockets for ingestion, block/object storage for the vault and databases, and CPU (optionally GPU) for rule-engine execution and local LLM inference.

4. Functional Requirements

Requirements are grouped by subsystem. Source: PS = derived from the problem statement; UD = derived from the ULPF Full Understanding & Reference Document; DER = derived from engineering analysis.

4.1 FR-1 — Ingestion Gateway

ID	Requirement	Source	Pri.
FR-1.1	The system shall ingest events from UDP and TCP Syslog listeners on configurable ports.	PS	High

ID	Requirement	Source	Pri.
FR-1.2	The system shall provide a synchronous single-event endpoint (POST /v1/convert) accepting an optional explicit rule_id.	UD	High
FR-1.3	The system shall provide a batch endpoint (POST /v1/jobs) accepting file upload with automatic detect-and-lock behaviour.	UD	High
FR-1.4	The system shall provide a streaming endpoint (POST /v1/sessions/{id}/events) carrying a session_id on every event with the same detect/lock/spot-check lifecycle as batch.	UD	High
FR-1.5	The system shall allocate a unique event_id to every ingested event before any processing.	PS	High
FR-1.6	The system shall bind every ingestion channel and every API key to exactly one registered log source/environment; a key scoped to one source shall not push or pull data for another.	UD	High
FR-1.7	The system shall record the receipt timestamp, transport, peer address and byte length for every ingested event.	DER	High
FR-1.8	The system shall apply backpressure or bounded buffering when downstream processing cannot keep pace, and shall never silently discard an accepted event.	DER	High
FR-1.9	API keys shall be generated on demand from the API & Keys screen, and shall be scoped, rotated or revoked per source; revocation shall affect only that source's integration.	UD	High
FR-1.10	Each API key shall carry an environment tag (at minimum Sandbox and Production).	UD	Medium

4.2 FR-2 — Detection & Rule Registry

ID	Requirement	Source	Pri.
FR-2.1	The system shall compute a format fingerprint for every incoming sample — a key-set hash for structured logs or a structural/delimiter signature for unstructured logs — before any parsing is attempted.	UD	High
FR-2.2	The system shall search only Active rules in the Rule Registry when auto-detecting a fingerprint.	UD	High
FR-2.3	The system shall never reject an event solely because its fingerprint matches no existing rule; a registry miss shall route to the Local Rule Authoring Agent during onboarding, or to unresolved storage at runtime if authoring is not in progress.	PS	High
FR-2.4	The system shall accept an explicit rule_id/version at call time; an explicit call shall still be validated per event rather than assumed correct.	UD	High
FR-2.5	For a batch job or streaming session, the system shall run full detection on the first 3–5 representative events and require all required fields to validate across samples before locking a rule.	UD	High
FR-2.6	The system shall lock the highest-confidence validated rule to the job/session and use it on a fast path for subsequent events.	UD	High
FR-2.7	The system shall run background spot-checks on a locked job/session at an adaptive rate: 1-in-50 immediately after a lock, 1-in-500 on a stable long-running fast path, 1-in-25 immediately after a mismatch, and shall unlock and return to full detection after repeated mismatches.	UD	High
FR-2.8	On a spot-check mismatch, the system shall flag the single event as unresolved and re-match it individually without failing the whole job or session.	UD	High

4.3 FR-3 — Local Rule Authoring Agent

ID	Requirement	Source	Pri.
FR-3.1	On a registry miss during onboarding, the system shall accept 3–5 representative samples and a target-schema selection (OCSF or ECS).	UD	High
FR-3.2	The system shall redact obvious secrets from samples before any sample is included in a prompt to the local model.	UD	High
FR-3.3	The Rule Authoring Agent shall run entirely within the air-gapped network using a local Qwen2.5-Coder-7B GGUF model through llama.cpp, and shall never be invoked outside onboarding.	UD	High
FR-3.4	The agent's system prompt shall instruct that all supplied samples are untrusted data, that in-sample instructions must never be followed, and that the only permitted output is declarative ULPF rule JSON.	UD	High
FR-3.5	The agent shall return JSON-only rule content comprising parser type, pattern/extraction, field mappings, required fields, masking hints and fingerprints.	UD	High
FR-3.6	The platform shall validate the candidate's JSON structure and shall reject forbidden content: executable code, shell commands, SQL, dynamic imports, URLs, credentials, filesystem paths or external actions.	UD	High
FR-3.7	The platform shall compile the candidate parser in a sandbox and run it against all supplied test samples before it may be saved.	UD	High
FR-3.8	On deterministic validation failure, the platform shall return structured errors to the agent for at most 2–3 retries.	UD	High
FR-3.9	A valid candidate shall be saved as Draft / Pending Review; the system shall never auto-activate a candidate.	UD	High
FR-3.10	The system shall require a human reviewer to approve a candidate before it may become Active.	PS	High
FR-3.11	The system shall record model/runtime provenance — model version, quantization, prompt template, candidate output, validation-run results and reviewer notes — against the final rule version.	UD	High
FR-3.12	The system shall verify local model and container image hashes at startup and shall record the model version and quantization in use.	UD	Medium
FR-3.13	The system shall bound sample count, input size, context length and output tokens for every onboarding invocation.	UD	Medium

4.4 FR-4 — Deterministic Parsing & Normalization

ID	Requirement	Source	Pri.
FR-4.1	The system shall load the exact matched or explicitly supplied rule version and run its declared parser type: regex (RE2), JSONPath, CEF, LEEF, Syslog, key-value or XML/XPath.	PS	High
FR-4.2	The system shall enforce a per-parse timeout and pattern/input size caps to bound catastrophic-backtracking and resource-exhaustion risk.	UD	High
FR-4.3	The system shall validate required fields, data types and target-schema compatibility before normalization; a failure shall not produce a normalized record.	PS	High
FR-4.4	The system shall normalize parsed fields into the ULPF canonical event model, then emit OCSF as the primary output and ECS as an adapter.	PS	High

ID	Requirement	Source	Pri.
FR-4.5	The system shall apply the rule's declarative masking policy (e.g. HMAC-SHA256) to configured sensitive fields in the normalized output only; the raw event remains unmasked and access-controlled.	UD	High
FR-4.6	The system shall place any parsed field with no safe common-schema mapping into unmapped_fields rather than discarding it.	PS	High
FR-4.7	On a validation failure, the system shall retain the raw data and return an explicit unresolved/rejected/schema_validation_failed outcome rather than silently emitting an incorrect normalized event.	UD	High
FR-4.8	The system shall never auto-apply a rule that is not Active; a Disabled rule shall be hard-rejected even when called explicitly.	UD	High

4.5 FR-5 — Raw Event Preservation

ID	Requirement	Source	Pri.
FR-5.1	The system shall persist the complete original event, byte-for-byte as received, to the Immutable Raw Evidence Layer before any parsing or transformation is applied.	PS	High
FR-5.2	The raw evidence layer shall be write-once; no component shall modify or overwrite a stored raw object.	PS	High
FR-5.3	The system shall compute and store a SHA-256 digest of each raw event at the time of storage.	UD	High
FR-5.4	The system shall verify the stored digest on retrieval and report any mismatch as an integrity failure.	DER	High
FR-5.5	The system shall store, alongside each raw event, the event_id, source identity, transport, receipt timestamp, original encoding and byte length.	UD	High
FR-5.6	The system shall provide retrieval of any raw event by event_id, returning content identical to what was received.	PS	High
FR-5.7	The system shall support a configurable retention window per source/tenant, with expiry recorded in the audit trail.	UD	Medium
FR-5.8	The system shall provide a verification utility that recovers every raw event in a defined corpus and reports the recoverability ratio.	DER	High
FR-5.9	If raw storage fails, the system shall buffer durably or reject/defer processing rather than emit a result implying a lossless guarantee.	UD	High

4.6 FR-6 — Traceability

ID	Requirement	Source	Pri.
FR-6.1	The system shall allocate a unique event_id to every ingested event at the moment of ingestion, before any processing.	PS	High
FR-6.2	Every normalized record shall carry the event_id, raw hash/reference, rule_id, rule_version, rule_hash and schema_version used to produce it.	UD	High
FR-6.3	The system shall resolve a normalized event to its originating raw event using the event_id.	PS	High
FR-6.4	The system shall resolve a raw event forward to the normalized event(s) derived from it.	PS	High
FR-6.5	The system shall expose an API returning the complete traceability record — raw reference, rule identity and schema version — for a given event_id.	PS	High

ID	Requirement	Source	Pri.
FR-6.6	The system shall record an immutable, append-only audit entry for every rule approval, lifecycle transition and configuration change, including actor identity and timestamp.	DER	High

4.7 FR-7 — Rule Registry and Lifecycle

ID	Requirement	Source	Pri.
FR-7.1	The system shall maintain a PostgreSQL Rule Registry holding rules, rule_versions, rule_fingerprints, rule_test_cases, rule_validation_runs, rule_approvals and rule_lifecycle_events.	UD	High
FR-7.2	The system shall enforce exactly one Active version per logical rule via a partial unique index.	UD	High
FR-7.3	The system shall support the lifecycle states Draft, Pending Review, Active, Deprecated, Disabled, Archived and Rejected, with auto-detect eligibility restricted to Active and explicit-call eligibility restricted to Active and Deprecated.	UD	High
FR-7.4	The system shall block deletion of rule versions; a retired version shall be archived instead.	UD	High
FR-7.5	The system shall block modification of parser or mapping content once a version is Active, Deprecated, Disabled or Archived; any change shall create a new version.	UD	High
FR-7.6	The system shall perform lifecycle transitions only through controlled stored procedures or service APIs, never through free update access.	UD	High
FR-7.7	A Disabled rule shall never be auto-reactivated; a new reviewed version shall be created instead.	UD	High
FR-7.8	The system shall enforce Row-Level Security for multi-tenant isolation; the application shall never run as a superuser or an RLS-bypassing owner.	UD	High
FR-7.9	The system should support export and import of rule and schema packages as files, to enable transfer between air-gapped deployments.	UD	Medium

4.8 FR-8 — Vendor Portal and Review Console

ID	Requirement	Source	Pri.
FR-8.1	The system shall provide Studio: a guided onboarding workflow from sample submission through fingerprinting, candidate review and approval.	UD	High
FR-8.2	The system shall present, for each candidate under review, the parser type, field mapping, required fields, masking hints, and the validation-run and test-case results.	UD	High
FR-8.3	The system shall provide a Jobs & Sessions screen showing a live per-event strip (detecting → locked → spot-checked) and the currently locked rule_id@version for each batch job or streaming session.	UD	High
FR-8.4	The system shall provide an API & Keys screen for generating, scoping, rotating and revoking source-scoped keys with environment tags.	UD	High
FR-8.5	The system shall provide a Log Review screen listing unresolved, ambiguous and format-drift events for manual triage.	UD	Medium
FR-8.6	The system shall record the identity of the approving/reviewing user and the timestamp for every rule decision.	DER	High

4.9 FR-9 — Downstream Delivery

ID	Requirement	Source	Pri.
FR-9.1	The system shall expose the ingestion endpoints defined in Section 5.2 of the Full Understanding Document: POST /v1/convert, POST /v1/jobs and POST /v1/sessions/{id}/events.	UD	High
FR-9.2	The system shall export normalized events to SIEM/data-lake consumers via Kafka-style asynchronous export and a REST/gRPC API.	PS	High
FR-9.3	The system shall store normalized events in ClickHouse for high-volume analytical queries.	UD	High
FR-9.4	The system should index normalized events into OpenSearch to support interactive SOC search and dashboards.	UD	Medium
FR-9.5	The system shall make unresolved and dead-lettered events retrievable with their failure diagnostics and raw reference.	DER	High
FR-9.6	The system shall ensure a normalized event delivered downstream always carries its event_id, rule_id, rule_version and schema_version.	DER	High

4.10 FR-10 — Deployment, Configuration and Administration

ID	Requirement	Source	Pri.
FR-10.1	The system shall be deployable as a set of containers started by a single Docker Compose command; Kubernetes orchestration is a later option.	PS	High
FR-10.2	The system shall install and operate with no outbound network access available at any stage, including the Rule Authoring Agent's model weights bundled at build time.	PS	High
FR-10.3	The system shall express sources, thresholds, rule profiles and output sinks in file-based, version-controllable configuration.	DER	High
FR-10.4	The system shall expose liveness, readiness and metrics endpoints for operational monitoring.	DER	Medium
FR-10.5	The system should enforce role-based access distinguishing viewer, approver and administrator roles.	DER	Medium
FR-10.6	The system should support running multiple stateless rule-engine workers scaled without code change.	PS	Medium
FR-10.7	The system shall write structured application logs of its own operation, separate from the event data it processes.	DER	High

5. Data Requirements

5.1 Principal Data Entities

ID	Entity	Description	Key attributes
DR-1	Raw Event	The original event exactly as received	event_id, source_id, received_at, transport, byte_length, sha256, storage_uri
DR-2	Source / Tenant	A registered origin of events with API-key scope	source_id, name, vendor, product, environment, status

ID	Entity	Description	Key attributes
DR-3	Rule	A logical stable rule identity	rule_key (e.g. cisco-asa-v1), owner, created_at
DR-4	Rule Version	An immutable versioned parser document	rule_id FK, version, parser_type, pattern/definition, field_mapping, masking, status, rule_hash
DR-5	Fingerprint	A candidate-selection signature for structured or unstructured formats	fingerprint_id, rule_version_id FK, type, value, required
DR-6	Test Case	Golden input/output tests, malformed cases and security regressions for a rule version	test_id, rule_version_id FK, input, expected_output
DR-7	Validation Run	Every compile/test result for a candidate or rule version	run_id, rule_version_id FK, result, duration, warnings, errors
DR-8	Approval	A human reviewer decision on a rule version	approval_id, rule_version_id FK, reviewer, decision, notes, decided_at
DR-9	Lifecycle Event	An append-only state-transition record	event_id, rule_version_id FK, from_state, to_state, actor, occurred_at
DR-10	Rule LLM Generation	Model/runtime provenance for an authoring-agent candidate	generation_id, rule_version_id FK, model_version, quantization, prompt_template, retries, candidate_output
DR-11	Ingestion Job / Session	Batch or streaming ingestion state and metrics	job_id/session_id, source_id FK, status, event_count, locked_rule_version_id
DR-12	Rule Lock	The rule selected and locked to a job/session	lock_id, job or session_id FK, rule_version_id FK, locked_at, mismatch_count
DR-13	Unresolved Event	Raw reference and diagnostic metadata for an event requiring triage	event_id FK, reason, diagnostic, queued_at
DR-14	Audit Event	An immutable record of a governance action	audit_id, actor, action, subject, before, after, occurred_at
DR-15	Normalized Event	The OCSF/ECS output event	event_id, rule_id, rule_version, rule_hash, schema_version, core_fields, unmapped_fields

5.2 Unified Event Model — Core Field Groups

The core schema is organised into OCSF-aligned object groups, with an ECS rendering profile over the same normalized event. The table below defines the minimum core surface required for the prototype.

Group	Representative fields	Purpose
metadata	event_id, schema_version, rule_id, rule_version, rule_hash	Identity and rule provenance of the normalized record
time	event_time, ingest_time	Original and ingestion-time temporal information
network	src_ip, src_port, dst_ip, dst_port, protocol	Transport-level characteristics of the observed activity
security	action, severity, event_category	Classification and outcome of the activity
source	vendor, product, device_type, format	The reporting device and the format it emitted

Group	Representative fields	Purpose
normalization	schema, rule_id, rule_version, rule_hash	Rule and schema traceability metadata
raw_reference	uri, sha256, stored_before_parse	Pointer and integrity reference to the preserved original
processing	status, mode, warnings	Outcome (normalized/unresolved/ambiguous/...) and processing mode (single/batch/session)
unmapped_fields	unmapped_fields.<vendor_key>	Vendor-specific data preserved without loss

5.3 Illustrative Normalized Event

```
{
  "event_id": "01JULPF...",
  "event_time": "2026-09-08T06:20:00Z",
  "ingest_time": "2026-09-08T06:20:00.231Z",
  "source": {"vendor": "Cisco", "product": "ASA", "device_type": "firewall", "format": "syslog"},
  "network": {"src_ip": "10.10.1.25", "src_port": 49821, "dst_ip": "203.0.113.10",
    "dst_port": 443, "protocol": "tcp"},
  "security": {"action": "allowed", "severity": 3, "event_category": "network_activity"},
  "normalization": {"schema": "ocsf", "rule_id": "cisco-asa-v1", "rule_version": 2, "rule_hash": "..."},
  "raw_reference": {"uri": "s3://ulpf-raw/...", "sha256": "...", "stored_before_parse": true},
  "processing": {"status": "normalized", "mode": "session_locked_rule", "warnings": []},
  "unmapped_fields": {"vendor_policy_id": "ACL-901"}
}
```

5.4 Data Integrity Requirements

- DR-I1 — A raw event, once written, shall never be altered by any process.
- DR-I2 — Every normalized event shall reference exactly one raw event by event_id.
- DR-I3 — A rule_version referenced by a normalized event shall remain retrievable, at minimum in Archived state, for as long as that event is retained.
- DR-I4 — Audit entries and lifecycle events shall be append-only.
- DR-I5 — A rule version's parser/mapping content shall be immutable once it leaves Draft/Pending Review.

5.5 Data Retention

Data class	Default retention	Configurable
Raw events	90 days	Yes — per source
Normalized events	90 days in ClickHouse/OpenSearch; exported files retained by the data platform	Yes
Rules and rule versions	Indefinite	No — required for reprocessing and audit
Fingerprints, test cases, validation runs	Indefinite	No
Rule LLM generation provenance	Indefinite	No
Audit entries	Indefinite	No
Unresolved / dead-lettered events	30 days	Yes

6. Non-Functional Requirements

6.1 Performance

ID	Requirement	Pri.
NFR-P1	The system shall process events on a locked-rule fast path that performs no fingerprinting beyond the periodic spot-check and never invokes the Rule Authoring Agent.	High
NFR-P2	The system shall achieve a sustained normalization throughput measured and reported on defined reference hardware and workload; the measured figure shall be published rather than estimated.	High
NFR-P3	The p95 normalization latency on the fast path shall be under 100 ms per event on reference hardware.	High
NFR-P4	The Rule Authoring Agent shall return a candidate for an interactive onboarding sample within 60 seconds on reference hardware.	High
NFR-P5	The complete onboarding workflow for an unseen source, from first sample to an Activated rule, shall complete within 10 minutes for the benchmark scenario.	High
NFR-P6	The Rule Authoring Agent shall execute on CPU without requiring a hardware accelerator, on a quantized GGUF build sized to available RAM.	High
NFR-P7	Query API responses for a bounded result page shall return within 2 seconds under nominal load.	Medium

6.2 Scalability

ID	Requirement	Pri.
NFR-S1	Rule-engine workers shall be stateless with respect to event content so additional workers can be added without coordination.	High
NFR-S2	The event stream shall be partitionable by source_id so ordering is preserved within a source while parallelism is achieved across sources.	Medium
NFR-S3	Adding a new log source or rule shall not require redeployment or restart of the processing tier.	High
NFR-S4	Storage growth shall be bounded by configurable retention rather than unbounded accumulation.	Medium
NFR-S5	The system shall document its measured single-node capacity and the architectural path to multi-node scale (Kafka/Redpanda partitioning), without claiming an unmeasured billions-of-events-per-day figure.	High

6.3 Reliability and Availability

ID	Requirement	Pri.
NFR-R1	No accepted event shall be lost; an event that cannot be processed shall appear in the unresolved/dead-letter store with diagnostics.	High
NFR-R2	The raw event shall be persisted and digested before acknowledging ingestion, so a crash after acknowledgement cannot lose data.	High
NFR-R3	Processing shall be idempotent with respect to event_id, so a retried event does not create duplicate normalized records.	High
NFR-R4	A spot-check mismatch shall degrade to per-event re-matching for the affected event only, never to failure of the whole job/session.	High

ID	Requirement	Pri.
NFR-R5	The system shall recover to a consistent state after an unclean shutdown without manual intervention.	Medium
NFR-R6	Target availability for the ingestion tier shall be 99.5% in a single-node prototype deployment.	Medium

6.4 Security

ID	Requirement	Pri.
NFR-SE1	All externally exposed HTTP interfaces shall require TLS.	High
NFR-SE2	API access shall require authentication (mTLS/API key); anonymous access shall not be permitted to event, raw or rule data.	High
NFR-SE3	The system shall enforce role-based authorisation distinguishing viewer, approver and administrator privileges.	Medium
NFR-SE4	The system shall not transmit log content or samples to any external service under any configuration, including for rule authoring.	High
NFR-SE5	The system shall treat all ingested content and all onboarding samples as untrusted input, and shall be resistant to injection through log content, including prompt injection directed at the Rule Authoring Agent.	High
NFR-SE6	A parsing rule shall never be permitted to contain executable code, shell commands, SQL, dynamic imports, URLs, credentials or network actions; this shall be enforced as a structural JSON-schema check, not a sandboxing exercise.	High
NFR-SE7	Every regex parser, and especially every LLM-drafted regex, shall be treated as a security-sensitive artefact: RE2 preferred, pattern/input size capped, and a per-parse timeout enforced.	High
NFR-SE8	Credentials and secrets shall be supplied through environment or secret files, never embedded in images or configuration committed to version control.	High
NFR-SE9	All governance actions (approvals, lifecycle transitions, key issuance/revocation) shall be recorded in an append-only audit trail.	High

6.5 Usability

- NFR-U1 — A rule/detection engineer familiar with log formats but unfamiliar with ULPF shall be able to complete a source onboarding without reading documentation beyond in-product guidance.
- NFR-U2 — A candidate rule shall be presented with its validation-run and test-case evidence, not as an unexplained approval prompt.
- NFR-U3 — The console shall make it visually unambiguous which rules are Active, Pending Review, Deprecated or Disabled, and which job/session events are locked, spot-checked or unresolved.
- NFR-U4 — Error messages shall state what failed, at which pipeline stage, and what the operator can do about it.

6.6 Maintainability and Portability

- NFR-M1 — Each subsystem shall be independently testable with defined inputs and outputs.
- NFR-M2 — Adding support for a new deterministic parser type shall require adding a parser module and registering it, with no change to the pipeline core.

- NFR-M3 — Adding a new target schema profile shall require adding an adapter, with no change to the rule engine.
- NFR-M4 — The system shall run on any host providing a compatible container runtime, without OS-specific code paths.
- NFR-M5 — All third-party dependencies and the GGUF model weights shall be pinned to explicit, hash-verified versions and vendored into the offline bundle.
- NFR-M6 — The system shall achieve at least 70% automated test coverage over the rule engine, fingerprinting and validation modules.

6.7 Compliance and Auditability

- NFR-C1 — For any normalized event presented as evidence, the system shall be able to produce the original raw record and the exact rule version that produced it.
- NFR-C2 — The integrity of a preserved raw event shall be demonstrable through a stored and re-verifiable SHA-256 digest.
- NFR-C3 — The system shall record which rule version and schema version was in force at the time each event was processed, so historical output is explicable.
- NFR-C4 — The system shall operate entirely within the customer's network boundary, supporting data-residency and sovereignty requirements, including for the Rule Authoring Agent.

7. Requirements Traceability

7.1 Problem Statement Coverage

PS capability	Satisfied by	Verified by
Preserve complete raw event data without loss	FR-5.1 – FR-5.9, FR-4.6, FR-4.7	TC-PRS-01 to TC-PRS-05
Extract and parse source-specific attributes	FR-4.1 – FR-4.3	TC-PRS-06 to TC-PRS-09
Normalize fields into a common taxonomy	FR-4.4, FR-4.6	TC-NRM-01 to TC-NRM-05
Maintain traceability between normalized and original events	FR-6.1 – FR-6.6	TC-TRC-01 to TC-TRC-06
Plug-and-play onboarding of new sources	FR-2.1 – FR-2.3, FR-3.1 – FR-3.13, FR-8.1 – FR-8.2	TC-ONB-01 to TC-ONB-08
Unified visibility across enterprise environments	FR-8.3 – FR-8.5, FR-9.3 – FR-9.4	TC-VIS-01 to TC-VIS-03
Efficient SIEM and Data Lake integration	FR-9.1 – FR-9.6	TC-OUT-01 to TC-OUT-05
AI/ML-ready security and operational analytics	FR-4.4, FR-9.3	TC-OUT-06
Reduced parser development effort	FR-3.1 – FR-3.13	TC-ONB-09
Deployable in an air-gapped network	FR-10.2, NFR-SE4	TC-DEP-01 to TC-DEP-03
Packaged in a container for platform independence	FR-10.1, NFR-M4	TC-DEP-01

7.2 Goal to Requirement Traceability

PRD goal	Requirements
G1 — Zero hand-written parser code	FR-3.1 – FR-3.13, FR-8.1 – FR-8.2, NFR-P4, NFR-P5
G2 — Complete recoverability of the original	FR-5.1 – FR-5.9, DR-I1, NFR-C2
G3 — Rule-hash provenance on every event	FR-6.1 – FR-6.6, DR-15, NFR-C1
G4 — Open-schema aligned output	FR-4.4, IR-2.5
G5 — No unreviewed rule ever reaches production	FR-3.8 – FR-3.10, FR-7.2 – FR-7.3, FR-7.6
G6 — Fully offline containerised operation, including the LLM	FR-10.1, FR-10.2, NFR-SE4, NFR-M4, NFR-M5
G7 — Surface format drift rather than silently re-interpret	FR-2.5 – FR-2.8

7.3 Verification Methods

Method	Applied to	Evidence produced
Unit test	Parsers, fingerprint computation, masking, rule-schema validation	Automated test report with coverage
Integration test	End-to-end pipeline from transport to normalized output, for each parser type	Pipeline test report per format family
Corpus test	The universality/format test matrix defined in Section 8	Coverage ratio against the test matrix
Rule validation harness	Every LLM-drafted candidate against its golden test samples	Candidate validation pass rate, retry count
Recoverability audit	Raw-event preservation	Recovered / ingested ratio across the full corpus
Benchmark	Throughput, latency and onboarding time	Measured figures with stated hardware and workload
Air-gap test	Offline installation and full operation, including onboarding	Egress monitor log showing zero outbound connections
Adversarial test	ReDoS payloads, prompt-injection samples, oversized/malformed input	Security regression report
Inspection / review	Documentation, configuration and audit-trail completeness	Review checklist sign-off

8. Validation Approach

8.1 Universality Test Matrix

Coverage is measured across a defined matrix of dimensions rather than an unbounded claim of "any format".

Dimension	Test categories
Format	Syslog RFC 3164/5424, JSON, CEF, LEEF, key-value text, CSV, delimited proprietary text
Source type	Firewall, router/switch, IDS/IPS, VPN gateway, WAF/proxy, cloud edge
Structure	Fully structured, semi-structured, free text with embedded fields, multi-line
Rule knowledge	Active-rule fast path, explicit rule_id call, registry miss requiring authoring
Rule lifecycle	No direct Active-rule edit, one Active version enforced, Disabled-rule rejection, Deprecated explicit use
Field composition	Standard fields only, standard plus vendor-specific fields, vendor-specific fields only
Evolution / drift	Field added, field renamed, template variant introduced mid-session
Transport	UDP Syslog, TCP Syslog, HTTP single event, batch file, streaming session
Adversarial input	Truncated event, malformed encoding, ReDoS payload, prompt-injection log, oversized field, embedded newline

8.2 Acceptance Criteria for the Prototype

- AC-1 — An event of a previously unseen format is ingested, its fingerprint computed, the Rule Authoring Agent drafts a candidate with parser type, mappings and required fields, and a reviewer approves it — with zero hand-written parser code.
- AC-2 — Every event ingested during the acceptance run is recoverable byte-identically from the Immutable Raw Evidence Layer, with digest verification passing.
- AC-3 — Every field present in a parsed event is present in the normalized event, either as a core field or under unmapped_fields.
- AC-4 — For any normalized event, the traceability API returns the raw reference, rule_id, rule_version, rule_hash and schema_version.
- AC-5 — A candidate that fails deterministic validation is retried at most 2–3 times, then held for manual authoring rather than activated.
- AC-6 — The full stack, including the Rule Authoring Agent, installs and operates on a host with no outbound network route, verified by an egress monitor.
- AC-7 — A locked batch job or streaming session that encounters a drifted event flags exactly that event as unresolved via spot-check, without failing the job/session, and unlocks after repeated mismatches.
- AC-8 — Throughput and latency figures are reported as measured values with the hardware and workload stated.

8.3 Reporting Discipline

Any metric not yet measured is reported as a target and labelled as such. Measured results replace targets only when produced by the benchmark harness on stated hardware against the stated corpus. This applies in particular to throughput, latency and rule-candidate validation pass rate, which are the figures most likely to be challenged during evaluation.